

Healthcare — GraphRAG with Neo4j

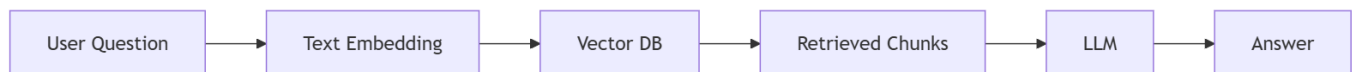
Diagram-Heavy Architecture README

This document explains the **GraphRAG architecture** used in the Citius Healthcare demos and capstone using **visual diagrams first**, then minimal text.

Audience: Architects, senior engineers, tech-savvy analysts

1 Why GraphRAG (Visual Motivation)

Traditional Vector RAG (Limitations)



Problems

- No explicit relationships
 - Poor reasoning over entities (patients, diagnoses, encounters)
 - Weak explainability
 - Hard to enforce grounding
-

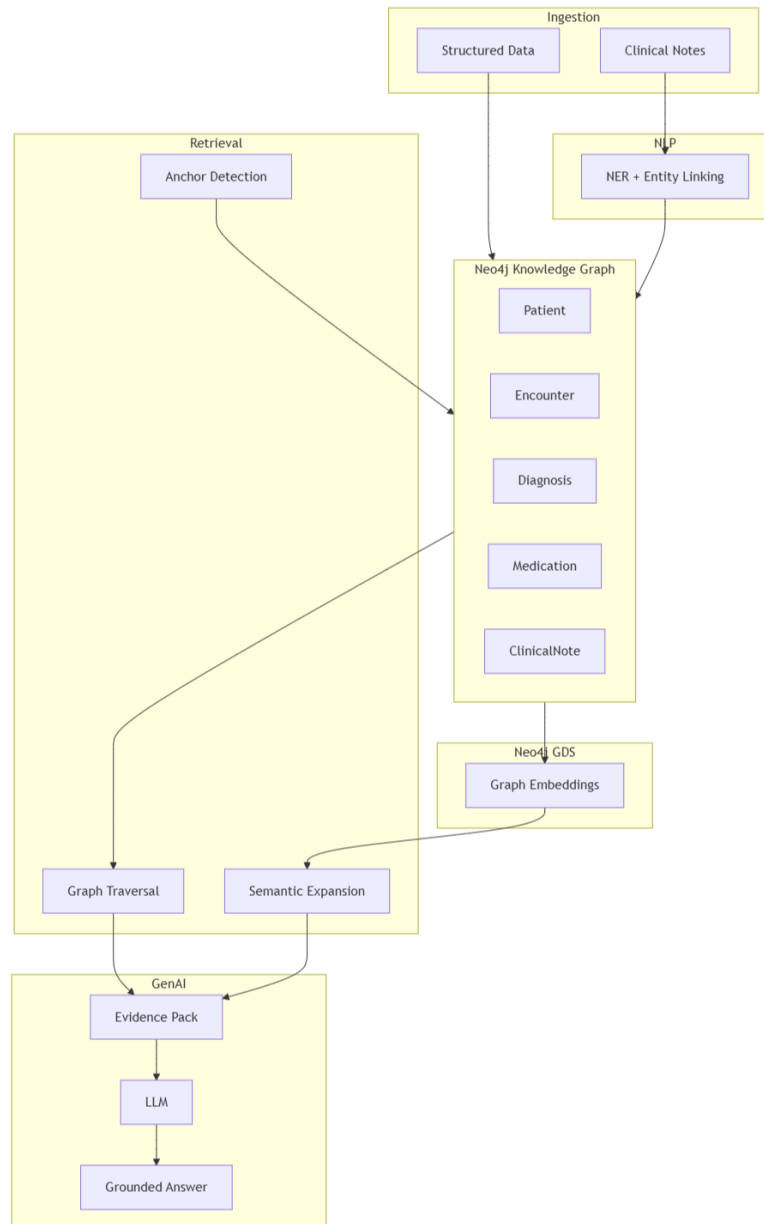
GraphRAG (Conceptual Upgrade)



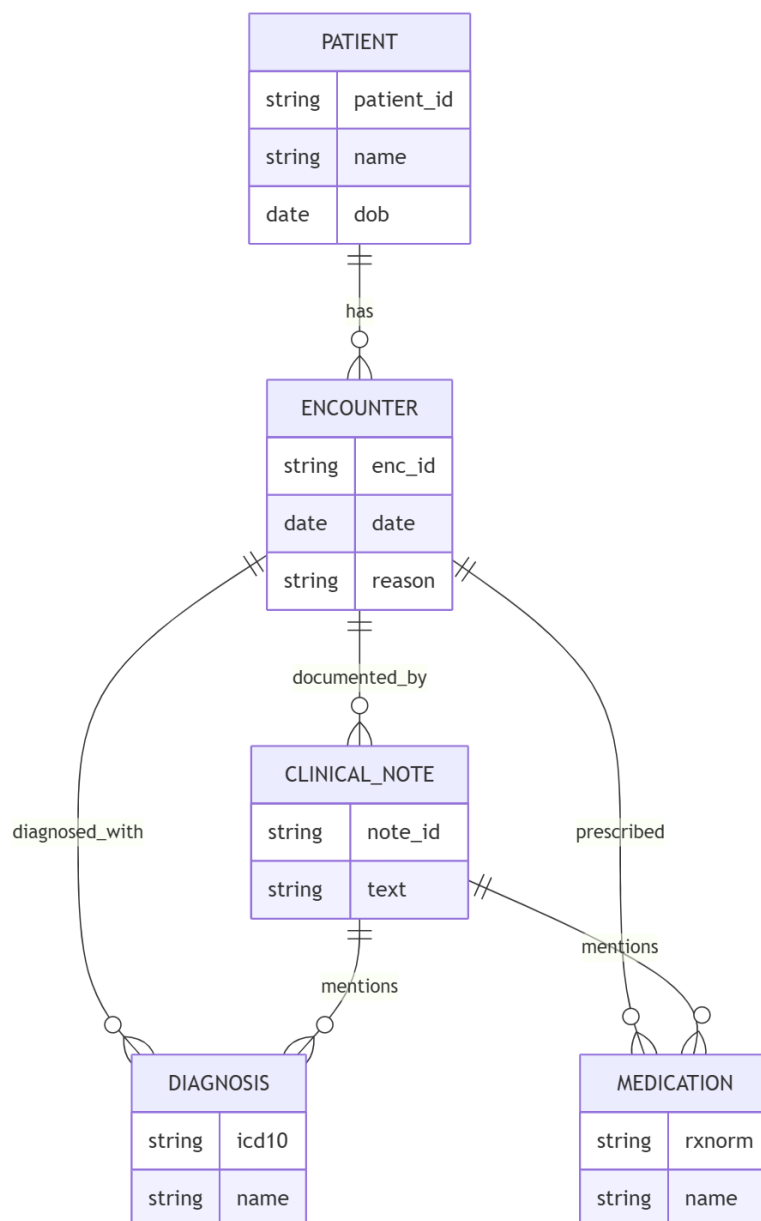
What changes

- Graph = first-class retrieval layer
 - Embeddings augment, not replace, structure
 - Evidence is explicit and citable
-

2 End-to-End GraphRAG Architecture



3 Knowledge Graph Data Model (Healthcare)



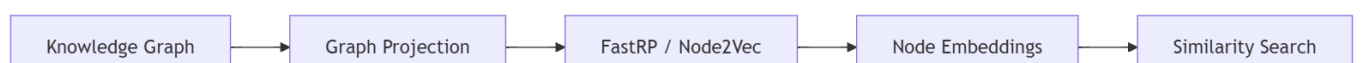
4 Entity Recognition & Linking Pipeline



Design principle

Prefer precision over recall
 Exact → Fuzzy → Semantic (fallback)

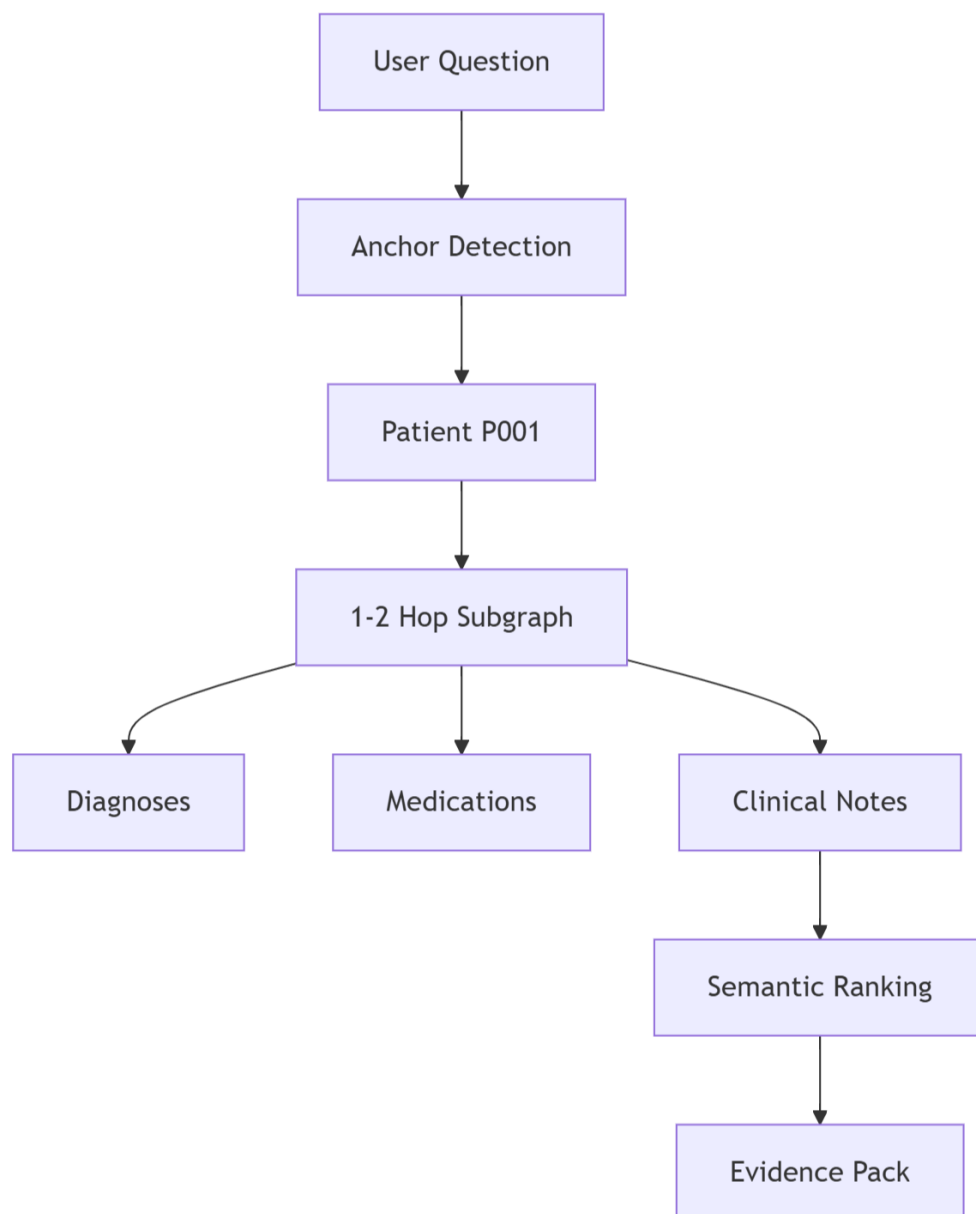
5 Graph Embeddings Layer



Used for

- Patient similarity
- Encounter similarity
- Semantic expansion of evidence

6 GraphRAG Retrieval (Evidence Pack)

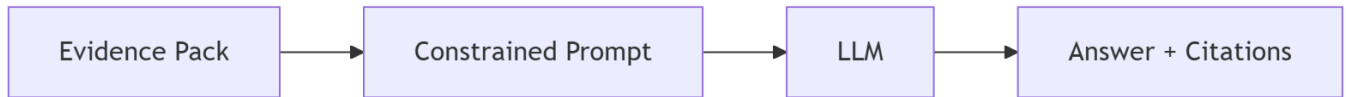


Evidence Pack Structure

```
{
  "anchors": { "patient_id": "P001" },
  "patient_context": {
    "diagnoses": [...],
    "medications": [...],
    "encounters": [...]
  },
  "note_semantic_hits": [...],
}
```

```
"similar_patients": [...]  
}
```

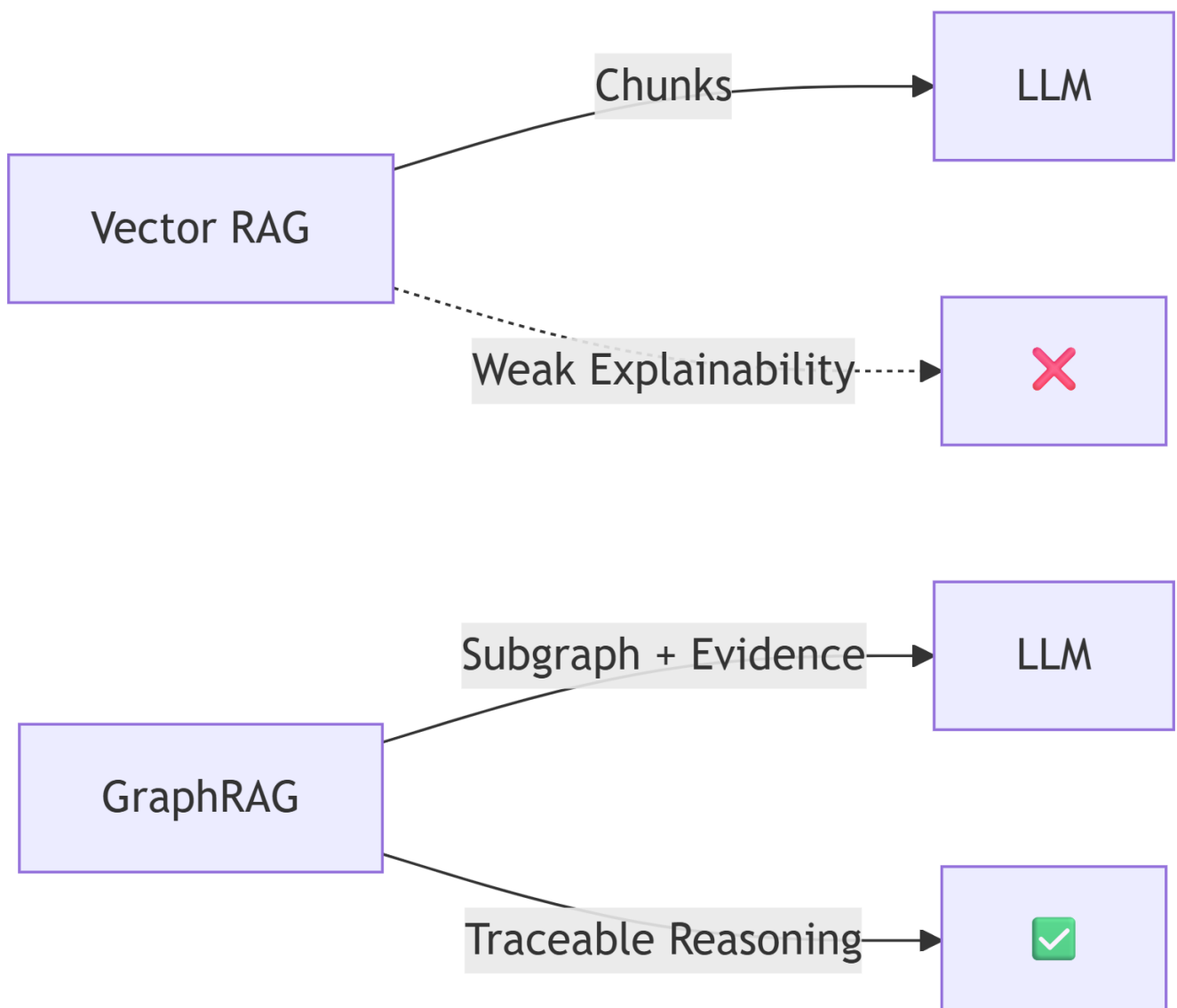
7 Grounded Answer Generation



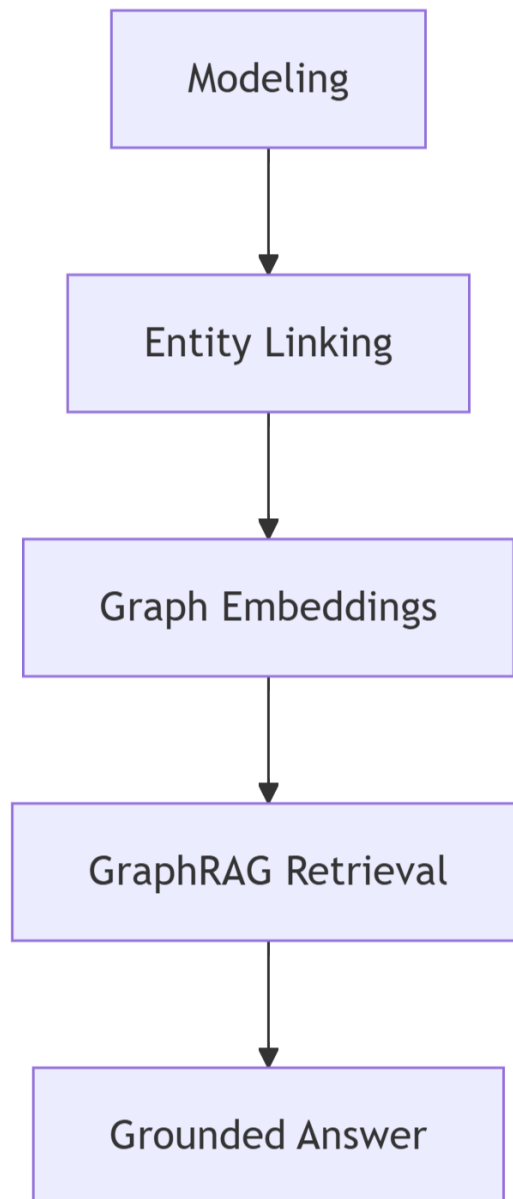
Rules enforced

- No knowledge outside evidence pack
- Explicit citations
- Admit insufficient evidence

8 GraphRAG vs Vector RAG (Visual Summary)



9 Capstone Alignment



Each participant notebook maps directly to **one block** in this pipeline.

Key Architectural Insight

GraphRAG is an architectural pattern, not a library.

Neo4j provides **structure + reasoning**, embeddings provide **semantic flexibility**, and LLMs provide **language intelligence**.
